

Contents

Speaker notes: Miłosz (slides 1–12)	1
Slide 1 — Title	1
Slide 2 — What we had to answer	1
Slide 3 — Quick vocabulary	2
Slide 4 — Two ways to deliver the same code	2
Slide 5 — What we built	3
Slide 6 — Meilisearch-inspired demo features	3
Slide 7 — Why not benchmark full Meilisearch?	3
Slide 8 — Shared API and parity checks	4
Slide 9 — Benchmark methodology	4
Slide 10 — Movie-search: cold start	4
Slide 11 — Movie-search: throughput and tail latency	5
Slide 12 — Movie-search: memory	5
Handover (after slide 12)	5
If you're running long	5

Speaker notes: Miłosz (slides 1–12)

About **7–8 minutes**. After slide 12, hand over to Damian.

The audience may not know WebAssembly or Spin — explain things in plain terms as you go. You do not need to sound like a manual.

Slide 1 — Title

Hi everyone — I'm Miłosz, and with Damian we're presenting our LSC project.

The short version: we took one small web service — a movie search API — and ran it in two different ways. One version is packaged as **WebAssembly**, which I'll unpack in a minute. The other is a normal **Docker container**, which is probably more familiar.

A microservice here just means a program that listens for HTTP requests and sends back JSON — like “search for space movies.” We cared about three practical questions: how fast does it **start** when it's cold, how many requests per second can it handle, and how much **memory** does it use.

The one thing I want you to remember from the title slide: we did **not** pit two different apps against each other. Same logic, two ways of shipping and running it.

Slide 2 — What we had to answer

The assignment asked us to deploy a microservice with **Spin** and **wasmtime** — that's the WebAssembly side — and compare it to an **OCI container**, which is what Docker builds and runs.

We turned that into a concrete question: is WebAssembly actually usable as the isolation layer behind serverless-style services? By serverless I don't mean "there is no server" — I mean workloads that spin up on demand, handle a request, and might scale in and out quickly.

To keep the comparison honest, we used the same Rust code paths, the same machine, and the same benchmark scripts for both sides. And when we talk about winners later, we always tie it back to **what kind of work** the endpoint was doing — not to slogans like "Wasm is always faster."

Slide 3 — Quick vocabulary

This slide is the mini-glossary. If any of this is new, that's fine — this is exactly why we put it here.

WebAssembly, or Wasm, is a portable binary format for programs. It started in the browser, but you can compile Rust — what we used — into Wasm and run it on a server too. Something still has to **execute** that binary; that's the runtime.

WASI is basically the list of things a Wasm program is allowed to ask the host for: files, HTTP, clocks, and so on. If it's not on the list, the program doesn't get it. That's the sandbox idea — tight, explicit permissions.

wasmtime is the open-source runtime that actually loads Wasm and enforces that sandbox. **Spin** sits on top: it's the toolkit we used to define HTTP routes, build the service, and run it locally. So when I say "Spin" in this talk, think "how we packaged our Wasm microservice," not a totally separate technology from wasmtime.

On the other side, **OCI** is the standard container image format — a filesystem snapshot plus meta-data that says "start this binary." **Docker** is what most people use to build and run those images. Under the hood you're looking at a normal OS process with namespaces and cgroups — a different isolation story than Wasm.

Slide 4 — Two ways to deliver the same code

So we wrote Rust once and shipped it two ways.

On the Spin side we compile to a Wasm **component** — in our case the target is called wasm32-wasip2. You get a relatively small artifact, and the only system access is what WASI allows. That model fits small HTTP handlers, edge functions, plugins — places where you want a hard sandbox and a narrow permission set.

On the Docker side we compile to a **native Linux binary**, drop it in an image with dependencies, and run it as a container. That path is what teams already use when they need rich native libraries, databases, or the usual ops tooling everyone knows.

Neither approach is inherently wrong. They're optimized for different constraints. Our job was to measure where each one looked good or bad — on **our** workloads.

Slide 5 — What we built

Here's the architecture on the slide.

In the middle is **one shared Rust library** — `movie-search-core` — with everything that actually defines search behavior: loading about forty-four thousand movies after deduplication, ranking, filters, facets, suggestions.

Wrapped around that are two thin adapters. Spin exposes it as a Wasm HTTP service on port **8080**. Docker exposes the same core as a native HTTP server on **8081**.

Below both sits a benchmark harness that runs smoke tests, checks that answers match, then does cold start, load, and memory measurements.

Our rule was simple: **same code first, measurements second**. If Spin and Docker returned different search results, we wouldn't trust any throughput graph.

Slide 6 — Meilisearch-inspired demo features

You might wonder why a benchmark project has filters, facets, and typo tolerance. That's because we took inspiration from **Meilisearch**, which is a real open-source search engine — a bit like a lightweight Elasticsearch.

We didn't embed full Meilisearch in the final benchmark — I'll explain why on the next slide — but we added similar **demo** features in the shared core: genre and year filters, facet counts, highlighted snippets in titles, optional fuzzy matching so a typo like `spce` can still find space-related movies, and a `/suggest` endpoint for autocomplete.

Important for fairness: the **old** benchmark request bodies didn't change. The fancy features are opt-in, and they live in the shared core once, so both runtimes behave the same. That makes the demo feel like a real product without gaming the performance comparison.

Slide 7 — Why not benchmark full Meilisearch?

We actually started with a simpler idea: run real Meilisearch in Docker and try to run it in Spin, apples to apples.

That broke down quickly. Meilisearch is built around LMDB — an embedded database — and memory-mapped files. It expects a normal native process on Linux with familiar filesystem behavior. Getting that whole stack to compile and behave identically inside `wasm32-wasi` isn't realistic. A hand-rolled Wasm substitute wouldn't be Meilisearch anymore.

So we kept Meilisearch as **design inspiration** and built our own deterministic in-memory search service with one shared core. What we're comparing is **how you isolate and run** the service — Wasm versus container — not two different search products.

Slide 8 — Shared API and parity checks

Before any load test, we proved the two sides were equivalent.

They share the same HTTP surface: health, version, stats, paginated movie listing, search, and suggest. We ran a script — `benchmarks/compare_results.sh` — with fixed queries like space, toy story, dark knight, romance, an empty query, and a heavier query with filters, facets, and highlights.

We only moved on when both sides agreed on the basics: same engine identity, same document count, same total hits, and the same **order** of movie IDs in the results, plus matching facets and suggestions. If the rankings diverged, comparing requests per second would be meaningless.

Slide 9 — Benchmark methodology

We ran the work in three passes.

First, **correctness**: unit tests, build Spin, build Docker, smoke tests, and the parity checks I just described.

Second, **performance**. We restarted services many times for cold start — about twenty runs — and we load-tested with different numbers of concurrent clients: 10, 50, 100, and 200. We used both the original search payloads and the enhanced ones. We tracked median latency and tail latency — p95 is the line where ninety-five percent of requests were faster; the slowest five percent still matter for users. Under load we also kept checking that responses stayed valid, not just fast.

Third, **memory**, idle and under load. One honest caveat: Docker gives you a container memory view; for Spin we measured host process RSS. Both are useful operationally, but they're not a perfect one-to-one comparison, and we say that in the report.

If you're running short on time here, you can compress this slide to three sentences — one per phase — but keep the memory caveat.

Slide 10 — Movie-search: cold start

Cold start is: you start the service from scratch — how long until it can actually serve traffic?

We measured two things. A cheap `/health` check — basically “are you up?” — and a first real search after recreating the service.

On `/health`, Docker was faster in our run — about twenty-one milliseconds median versus about one hundred forty-six for Spin. But on the first real search, Spin was faster on median and on p95 — around three hundred seventy milliseconds versus about four hundred thirty-two for Docker.

The lesson isn't “Docker wins cold start” or “Spin wins cold start.” It's that a fast health endpoint doesn't tell the whole story. If you care about serverless, measure the path your user actually hits.

Slide 11 — Movie-search: throughput and tail latency

These charts are requests per second and tail latency under load.

With an **empty** search query, Docker shot ahead — thousands of requests per second versus dozens for Spin at low concurrency. That’s mostly because almost no search work happens; you’re measuring a very cheap path. Useful, but it’s not the same as stressing ranking logic.

With a real query like space, Spin led at low concurrency, and at very high concurrency both sides started timing out — that’s saturation, not a clear victory for either.

With the **enhanced** query — filters, facets, highlights — Spin was ahead at low concurrency again, but tails grew ugly as we pushed concurrency up.

So the pattern is: who wins depends on how expensive the query is and how hard you push the service. Don’t read one bar and walk away with a universal rule.

Slide 12 — Movie-search: memory

This slide is the one that surprises people if they’ve only read hype about Wasm.

You sometimes hear “Wasm is always lighter than containers.” For our movie-search setup, that’s not what we saw. Docker sat around thirty-five megabytes idle and stayed modest under load. Spin was higher idle and peaked much higher under load in this configuration — hundreds of megabytes at the top.

Wasm still gives you a different isolation model — sandbox, explicit capabilities. It just doesn’t automatically mean lower resident memory on every host and every runtime version.

So our takeaway from movie-search: pick your deployment model **after** you measure **your** endpoints the way **you** measure them. Same code first, measurements second.

Handover (after slide 12)

That wraps my part — the movie-search workload, one shared core, Spin on 8080, Docker on 8081, parity before benchmarks.

Damian will take you through the second workload, file-tools — everyday JSON and image operations — and then pull everything together with recommendations and conclusions. Thanks.

If you’re running long

- Slide 3: keep WebAssembly, Spin, and Docker; mention WASI and wasmtime in one breath.
- Slide 9: one sentence per phase, but don’t skip the memory caveat.
- Spend your time on slides 10–12 — that’s where the story lands.